



COMPUTER SCIENCE

CS-305L: Parallel & Distributed Computing Lab

SUBMITTED BY

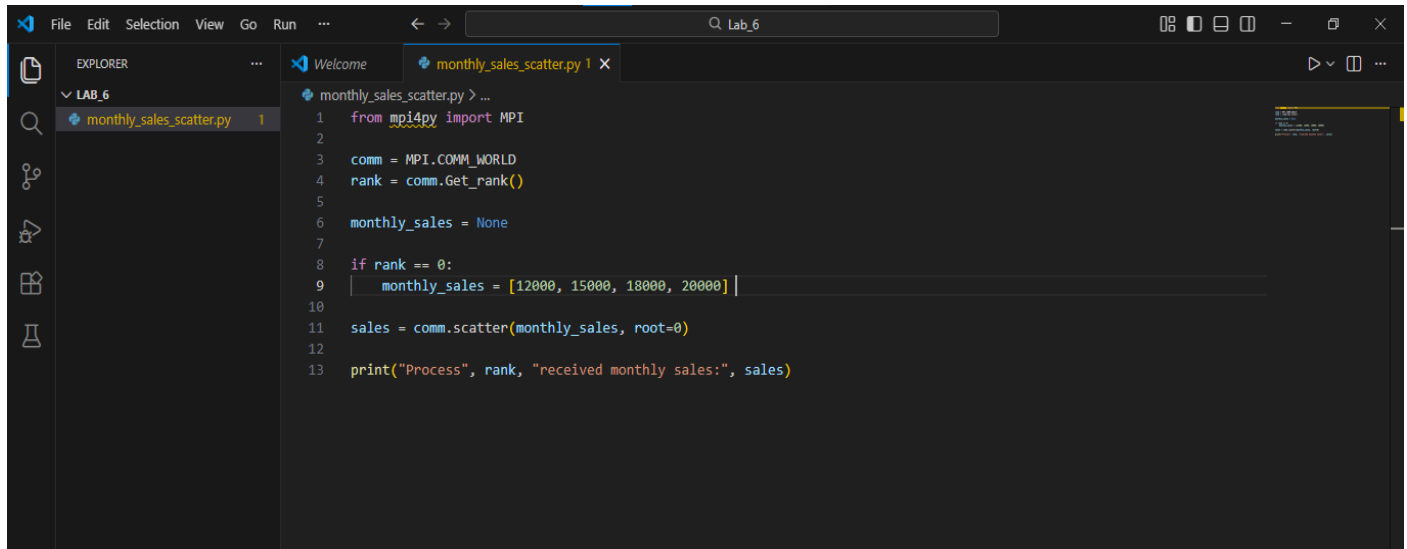
NAME : RIFAQ AJMAL
REG NO : 23MDBCS 435
SECTION : CS-A
SEMESTER : 6th

Date: 1/3/2026

Lab 6 (5a)

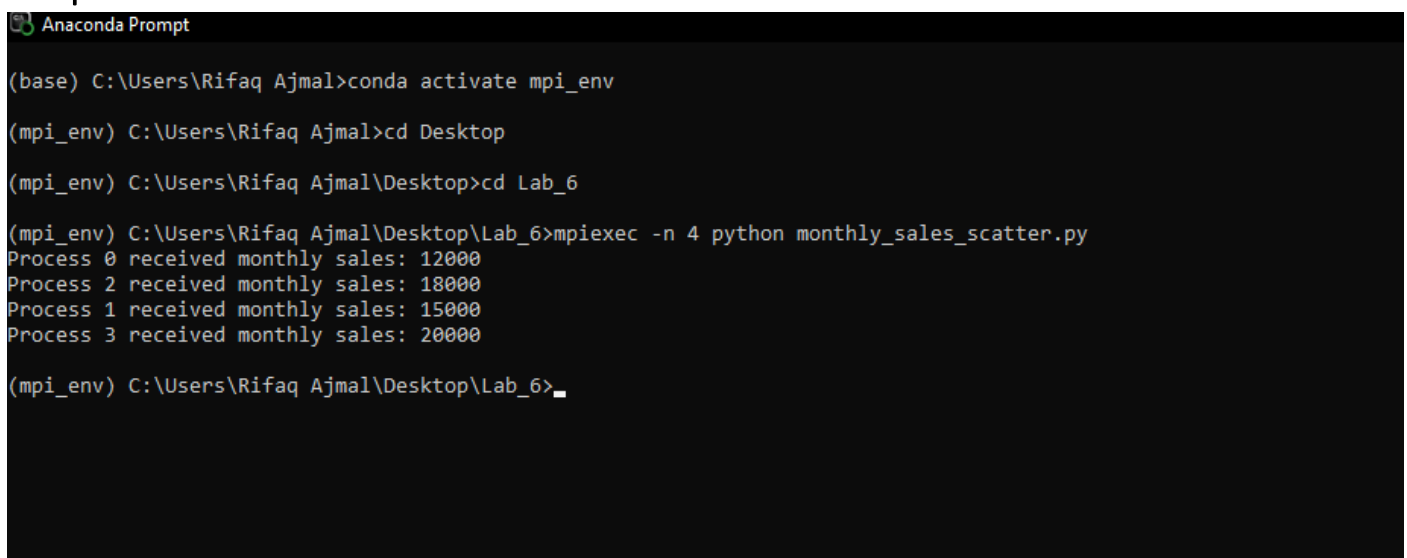
Task 1

Modify sales data distribution to monthly reports (MPI_Scatter)



```
1 from mpi4py import MPI
2
3 comm = MPI.COMM_WORLD
4 rank = comm.Get_rank()
5
6 monthly_sales = None
7
8 if rank == 0:
9     monthly_sales = [12000, 15000, 18000, 20000]
10
11 sales = comm.scatter(monthly_sales, root=0)
12
13 print("Process", rank, "received monthly sales:", sales)
```

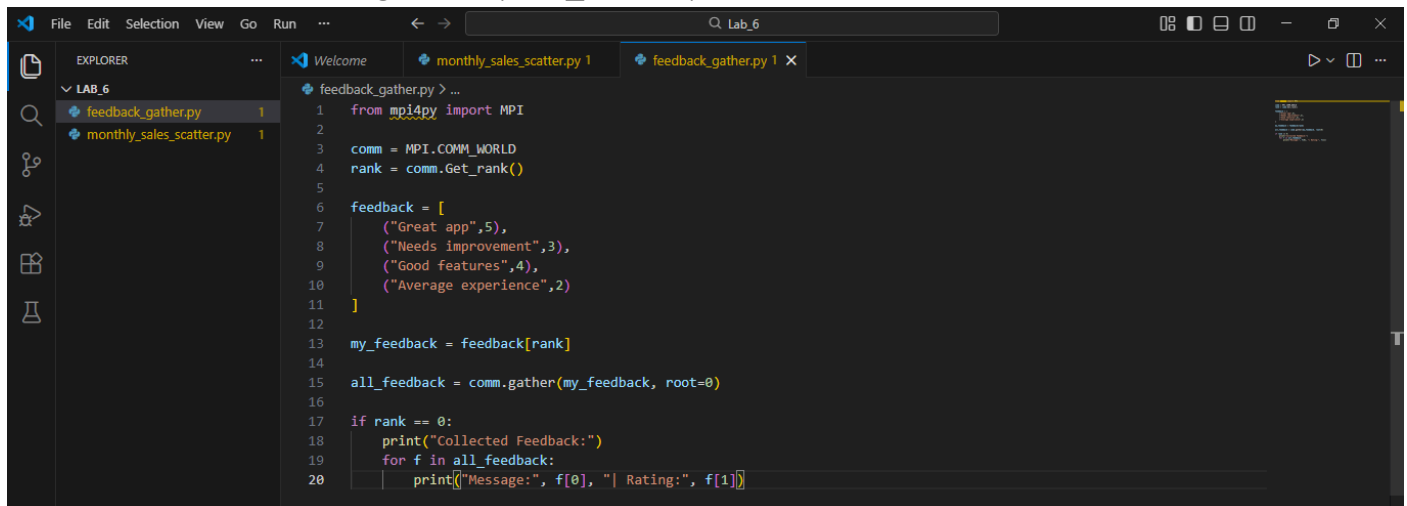
Output



```
(base) C:\Users\Rifaq Ajmal>conda activate mpi_env
(mpi_env) C:\Users\Rifaq Ajmal>cd Desktop
(mpi_env) C:\Users\Rifaq Ajmal\Desktop>cd Lab_6
(mpi_env) C:\Users\Rifaq Ajmal\Desktop\Lab_6>mpiexec -n 4 python monthly_sales_scatter.py
Process 0 received monthly sales: 12000
Process 2 received monthly sales: 18000
Process 1 received monthly sales: 15000
Process 3 received monthly sales: 20000
(mpi_env) C:\Users\Rifaq Ajmal\Desktop\Lab_6>_
```

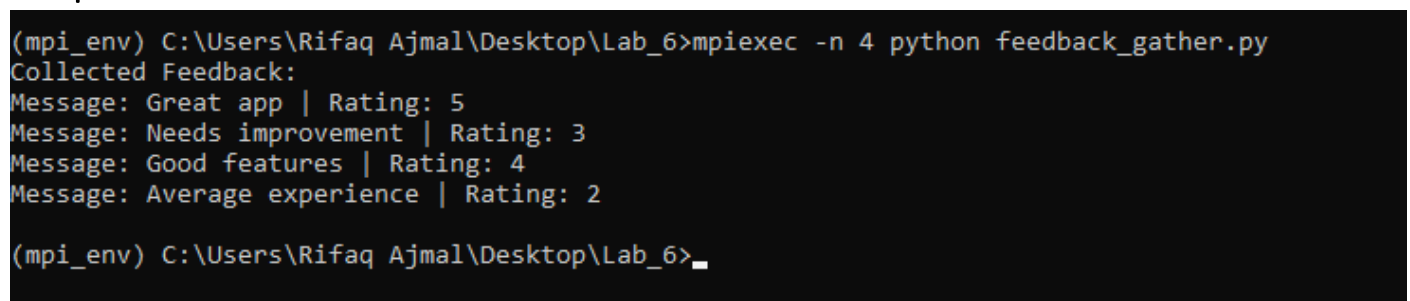
Task 2

User feedback with rating + text (MPI_Gather)



```
1 from mpi4py import MPI
2
3 comm = MPI.COMM_WORLD
4 rank = comm.Get_rank()
5
6 feedback = [
7     ("Great app",5),
8     ("Needs improvement",3),
9     ("Good features",4),
10    ("Average experience",2)
11]
12
13 my_feedback = feedback[rank]
14
15 all_feedback = comm.gather(my_feedback, root=0)
16
17 if rank == 0:
18     print("Collected Feedback:")
19     for f in all_feedback:
20         print(f"Message:", f[0], "| Rating:", f[1])
```

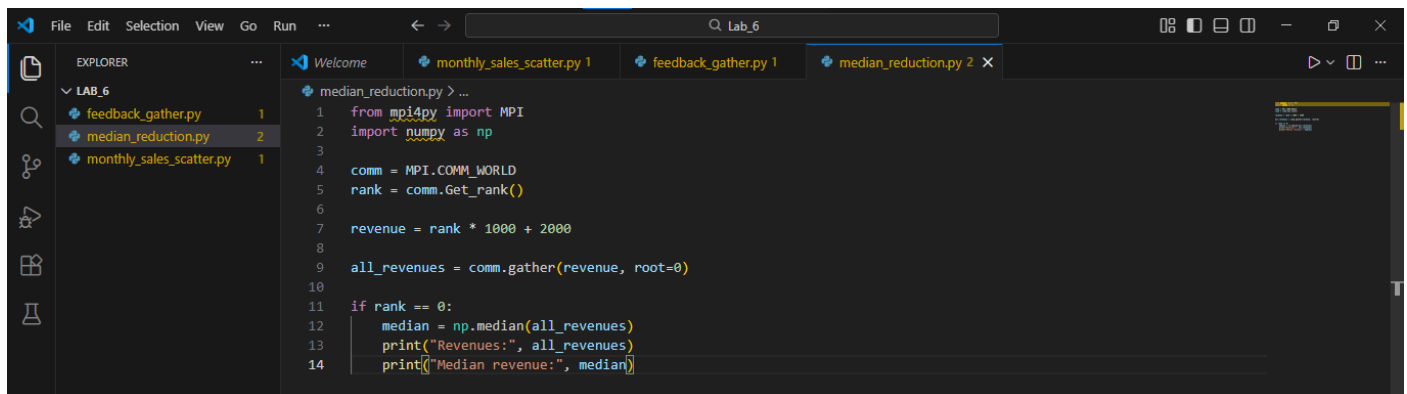
Output



```
(mpi_env) C:\Users\Rifaq Ajmal\Desktop\Lab_6>mpirun -n 4 python feedback_gather.py
Collected Feedback:
Message: Great app | Rating: 5
Message: Needs improvement | Rating: 3
Message: Good features | Rating: 4
Message: Average experience | Rating: 2
(mpi_env) C:\Users\Rifaq Ajmal\Desktop\Lab_6>
```

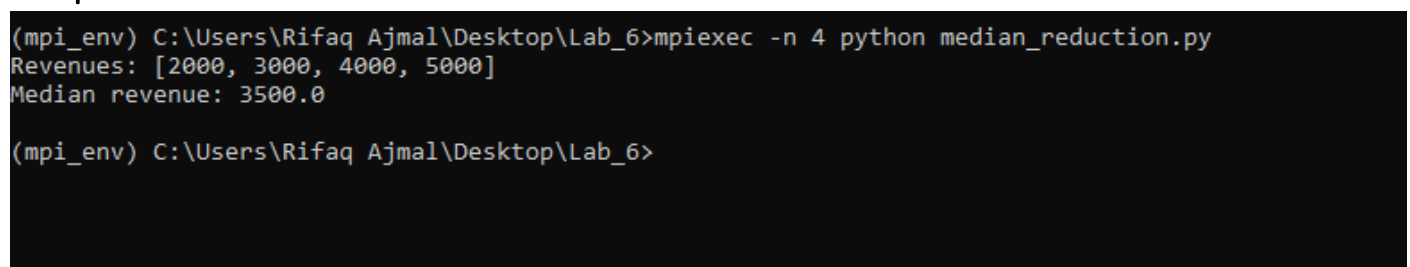
Task 3

Custom reduction to find median revenue.



```
1 from mpi4py import MPI
2 import numpy as np
3
4 comm = MPI.COMM_WORLD
5 rank = comm.Get_rank()
6
7 revenue = rank * 1000 + 2000
8
9 all_revenues = comm.gather(revenue, root=0)
10
11 if rank == 0:
12     median = np.median(all_revenues)
13     print("Revenues:", all_revenues)
14     print("Median revenue:", median)
```

Output



```
(mpi_env) C:\Users\Rifaq Ajmal\Desktop\Lab_6>mpirun -n 4 python median_reduction.py
Revenues: [2000, 3000, 4000, 5000]
Median revenue: 3500.0
(mpi_env) C:\Users\Rifaq Ajmal\Desktop\Lab_6>
```

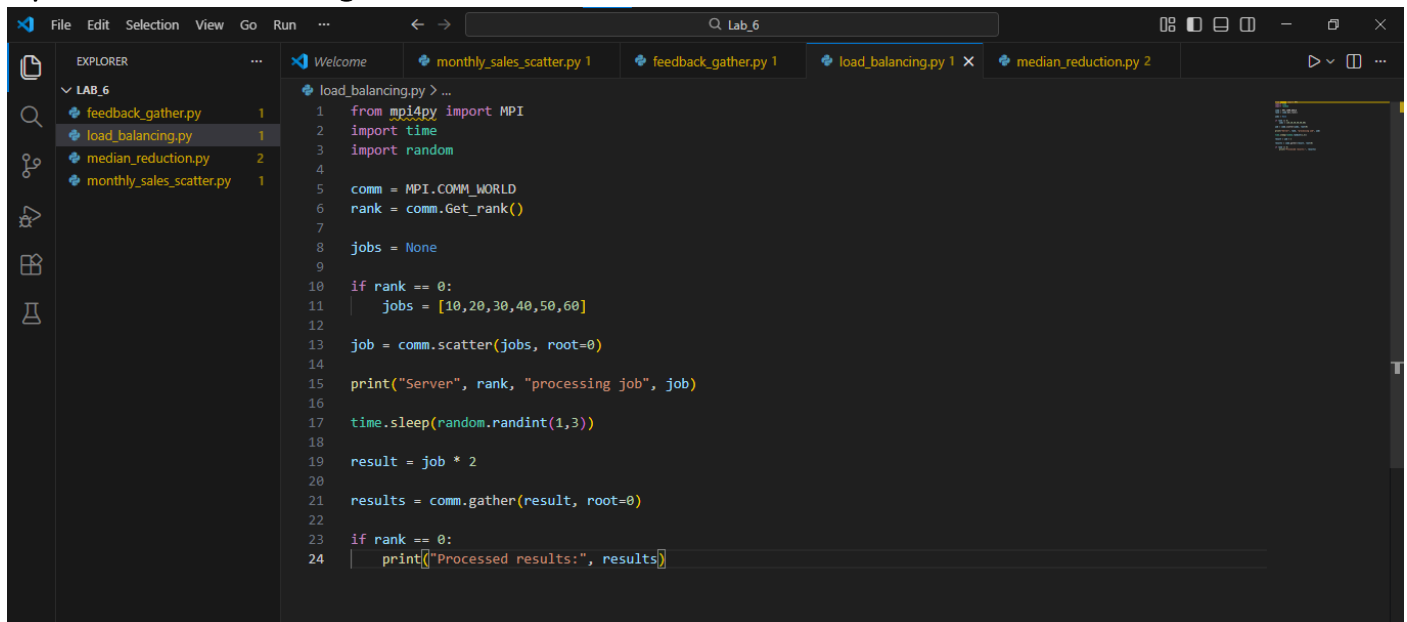
Task 4

If some servers have delayed responses

If some servers respond slowly, synchronization methods such as `MPI_Barrier` or timeout handling can be used. The system can wait for all servers before calculating results or ignore delayed responses after a certain time limit. Load balancing can also help distribute work more efficiently.

Task 5 (Challenge)

Dynamic load balancing



The screenshot shows a Visual Studio Code editor with a project named 'LAB_6'. The Explorer pane on the left lists four files: `feedback_gather.py` (1 instance), `load_balancing.py` (1 instance), `median_reduction.py` (2 instances), and `monthly_sales_scatter.py` (1 instance). The `load_balancing.py` file is open in the editor, showing the following code:

```
1 from mpi4py import MPI
2 import time
3 import random
4
5 comm = MPI.COMM_WORLD
6 rank = comm.Get_rank()
7
8 jobs = None
9
10 if rank == 0:
11     jobs = [10, 20, 30, 40, 50, 60]
12
13 job = comm.scatter(jobs, root=0)
14
15 print("Server", rank, "processing job", job)
16
17 time.sleep(random.randint(1, 3))
18
19 result = job * 2
20
21 results = comm.gather(result, root=0)
22
23 if rank == 0:
24     print("Processed results:", results)
```

Output

```
(mpi_env) C:\Users\Rifaq Ajmal\Desktop\Lab_6>mpirun -n 6 python load_balancing.py
Server 1 processing job 20
Server 3 processing job 40
Server 2 processing job 30
Server 5 processing job 60
Server 4 processing job 50
Server 0 processing job 10
Processed results: [20, 40, 60, 80, 100, 120]

(mpi_env) C:\Users\Rifaq Ajmal\Desktop\Lab_6>
```